

Smart Contract Audit Report

Contract: MyContract

Date: 2025-07-08 22:59:18

Risk Summary

Risk Score: 20/100

Risk Level: Low

Vulnerabilities Found: 2

Vulnerability Findings

1. Unchecked arithmetic operation

Severity: Unknown

Location: a + b

Description: Operation 'a + b' might cause overflow/underflow. Consider using safe math checks.

2. Missing validation in onTransfer

Severity: Unknown

Location: onTransfer

Description: onTransfer() should validate sender and amount to prevent exploit.

AI-Powered Analysis

Smart Contract Audit Summary for MyContract

Vulnerability 1: Unchecked Arithmetic Operation

Severity: Medium

Impact: Potential overflow or underflow of variables `a` and `b` in the expression `a + b`, leading to unexpected behavior or incorrect results.

Potential Exploit Paths: An attacker could manipulate the values of `a` and `b` to cause an overflow or underflow, potentially allowing them to drain the contract's funds or manipulate the contract's state.

Mitigation Strategies:

+ Use safe math libraries, such as OpenZeppelin's SafeMath, to perform arithmetic operations. These

libraries include built-in checks for overflow and underflow.

- + Implement explicit checks for overflow and underflow before performing the arithmetic operation.
- + Consider using uint256 variables instead of uint, as they provide a larger range of values and reduce the likelihood of overflow.

****Advice for Developers:**** When performing arithmetic operations, always consider the potential for overflow or underflow. Use safe math libraries or implement explicit checks to ensure the correctness and security of your contract.

****Vulnerability 2: Missing Validation in onTransfer****

* ****Severity:**** High

* ****Impact:**** The lack of validation in the `onTransfer` function allows an attacker to manipulate the `sender` and `amount` variables, potentially leading to unauthorized transfers or exploits.

* ****Potential Exploit Paths:**** An attacker could call the `onTransfer` function with malicious input, such as a zero address as the sender or an excessively large amount, to drain the contract's funds or manipulate the contract's state.

* ****Mitigation Strategies:****

- + Implement input validation for the `sender` and `amount` variables in the `onTransfer` function. Ensure that the sender is a valid address and the amount is within a reasonable range.
- + Use the `require` statement to enforce these validation checks and revert the transaction if the input is invalid.
- + Consider implementing access control mechanisms, such as only allowing authorized addresses to call the `onTransfer` function.

****Advice for Developers:**** Always validate user input in your contract's functions, especially those that involve sensitive operations like transfers. Use the `require` statement to enforce these validation checks and ensure the security of your contract.

****Recommendations:****

- * Address these vulnerabilities as soon as possible to prevent potential exploits.
- * Perform thorough testing and code reviews to ensure the security and correctness of the contract.

* Consider engaging a third-party security expert or conducting a comprehensive smart contract audit to identify and address any additional vulnerabilities.

By addressing these vulnerabilities and implementing the recommended mitigation strategies, you can significantly improve the security and reliability of your MyContract smart contract.

Generated by DeFi Sentinel